Employee Management System

Name : Muhammad Abdul Rehman

Reg # : 4767/bsse/foc/f23

Section: A

Course: Object Oriented Paradigm

Submitted to : Sir Qaiser Afzal

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip> // For formatted output

using namespace std;

// Employee Class
class Employee {
    string name;
    string role;
    double salary;

public:
    // Constructor
    Employee(const string& name, const string& role,
double salary)
        : name(name), role(role), salary(salary) {}
```

```
    // Getters
    string getName() const { return name; }
    string getRole() const { return role; }
    double getSalary() const { return salary; }

    // Setters
    void setRole(const string& newRole) { role =
newRole; }
    void setSalary(double newSalary) { salary =
newSalary; }

    // Display Employee Details
    void displayDetails() const {
        cout << left << setw(15) << name
             << setw(15) << role
             << "$" << salary << endl;
    }
```

```
};

// Employee Management System
class EMS {
    vector<Employee> employees;

public:
    // Add a new employee
    void addEmployee(const string& name, const string& role, double salary) {
        employees.emplace_back(name, role, salary);
        cout << "Employee \"" << name << "\" added successfully.\n";
    }

    // Remove an employee by name
    void removeEmployee(const string& name) {
```

```
        for (auto it = employees.begin(); it != employees.end(); ++it) {
            if (it->getName() == name) {
                employees.erase(it);
                cout << "Employee \"" << name << "\" removed successfully.\n";
                return;
            }
        }
        cout << "Employee \"" << name << "\" not found.\n";
    }

    // Update employee details
    void updateEmployee(const string& name, const string& newRole, double newSalary) {
        for (auto& emp : employees) {
            if (emp.getName() == name) {
```

```
            emp.setRole(newRole);
            emp.setSalary(newSalary);
            cout << "Employee \"" << name << "\" updated successfully.\n";
            return;
        }
    }
    cout << "Employee \"" << name << "\" not found.\n";
}

// View the list of employees
void viewEmployees() const {
    if (employees.empty()) {
        cout << "No employees to display.\n";
        return;
    }
```

```
        cout << "\nEmployee List:\n";
        cout << left << setw(15) << "Name"
             << setw(15) << "Role"
             << "Salary\n";
        cout << string(40, '-') << endl;

        for (const auto& emp : employees) {
            emp.displayDetails();
        }
    }
};

// Menu-driven interface
void displayMenu() {
    cout << "\n--- Employee Management System ---
\n";
    cout << "1. Add Employee\n";
    cout << "2. Remove Employee\n";
```

```
    cout << "3. Update Employee\n";
    cout << "4. View Employees\n";
    cout << "5. Exit\n";
    cout << "Enter your choice: ";
}

int main() {
    EMS ems;
    int choice;

    do {
        displayMenu();
        cin >> choice;

        switch (choice) {
            case 1: {
                string name, role;
```

```
        double salary;

        cout << "Enter employee name: ";
        cin.ignore();
        getline(cin, name);

        cout << "Enter role: ";
        getline(cin, role);

        cout << "Enter salary: ";
        cin >> salary;

        ems.addEmployee(name, role, salary);
        break;
    }
    case 2: {
        string name;
```

```
            cout << "Enter the name of the employee
to remove: ";
            cin.ignore();
            getline(cin, name);
            ems.removeEmployee(name);
            break;
        }
        case 3: {
            string name, newRole;
            double newSalary;

            cout << "Enter the name of the employee
to update: ";
            cin.ignore();
            getline(cin, name);

            cout << "Enter new role: ";
            getline(cin, newRole);
```

```
            cout << "Enter new salary: ";
            cin >> newSalary;

            ems.updateEmployee(name, newRole,
newSalary);
            break;
        }
        case 4:
            ems.viewEmployees();
            break;
        case 5:
            cout << "Exiting the program.
Goodbye!\n";
            break;
        default:
            cout << "Invalid choice! Please try
again.\n";
```

```
        }
    } while (choice != 5);

    return 0;
}
```

Output: